MILESTONE 1: Model Selection and Architecture – LegalEase

In this milestone, we focus on selecting the optimal generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system architecture and set up the development environment to support document generation, editing, and exporting in various formats (.docx, .pdf, .txt).

Activity 1.1: Research and Select the Appropriate Generative AI Model

1. **Understand the Project Requirements**
    - LegalEase must generate structured legal documents (e.g., contracts, NDAs, agreements) based on user inputs such as:
        - Document type
        - Parties involved
        - Terms and conditions
        - Effective dates
2. **Explore Gemini Model Documentation**
    - Visit Google Generative AI Documentation to evaluate available Gemini models.
    - Review capabilities of gemini-1.5-pro, especially:
        - Long context window (up to 1 million tokens)
        - Structured prompting with multi-part content
        - Fast generation speed and legal-domain compatibility
3. **Evaluate Model Performance**
    - Benchmarks indicate that gemini-1.5-pro is capable of handling complex formatting, reasoning, and structured text generation, making it suitable for legal content generation.
4. **Select the Optimal Model**

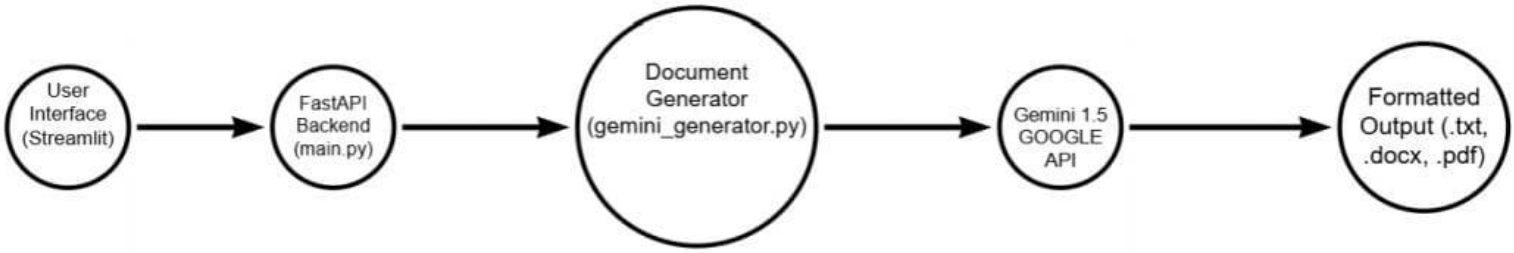
    Selected: gemini-1.5-pro

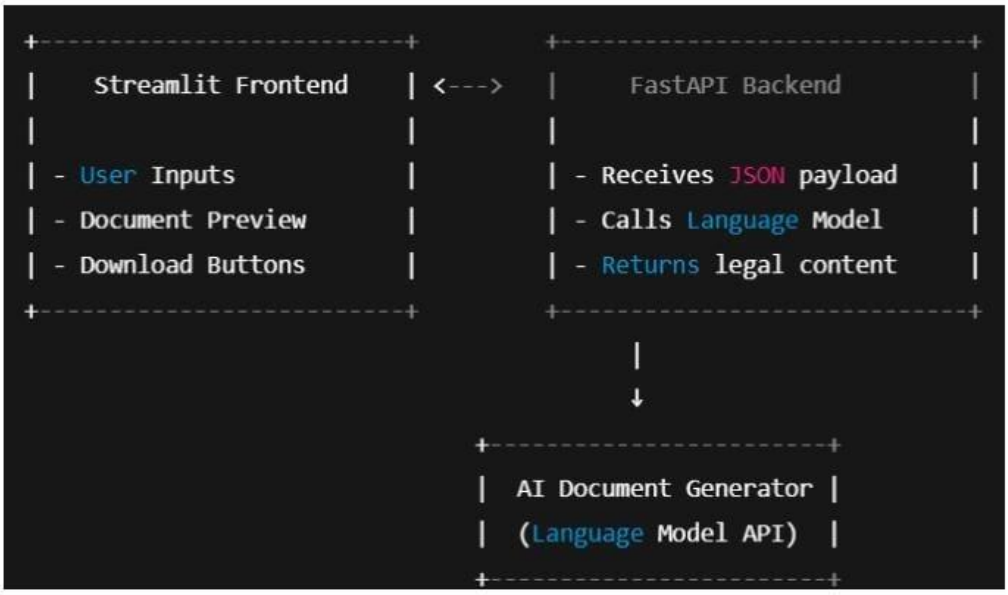
        - Best suited for drafting formal documents with clear sections and legal jargon.
        - Capable of understanding and generating clauses based on user inputs dynamically.

Activity 1.2: Define the Architecture of the Application

1. Architectural Diagram

- Components:
    - **Frontend**: Streamlit for user interaction (document input, preview, editing, downloads)
    - **Backend**: FastAPI app (routes input to Gemini model and returns generated content)
    - **AI Core**: Gemini integration for content generation
    - **Formatting Modules**: DOCX, PDF, TXT generators





2. Frontend Functionality

- Users provide inputs via:
  - TextInput for document type and dates
  - TextArea for parties and terms
- Buttons:
  - "Generate Document"
  - "Edit Document"
  - "Download as TXT / DOCX / PDF"
- Styled HTML preview of the generated legal document

3. Backend Responsibilities

- Expose POST /generate endpoint (via FastAPI)
- Accept JSON input (document_type, parties, terms, dates)
- Route to GeminiDocumentGenerator
- Return structured document text

4. AI Integration Points

- GeminiDocumentGenerator forms a structured prompt and sends it to Gemini via generate_content
- Receives generated text and returns to FastAPI
- Frontend displays and allows editing or exporting

Activity 1.3: Set Up the Development Environment

1. **Install Python and Pip**
   Ensure Python 3.10+ is installed along with pip for dependency management.
2. **Create a Virtual Environment**
3. **Install FastAPI and Uvicorn (Backend Server)**

pip install fastapi uvicorn

4. **Install Streamlit (Frontend UI)**

pip install streamlit

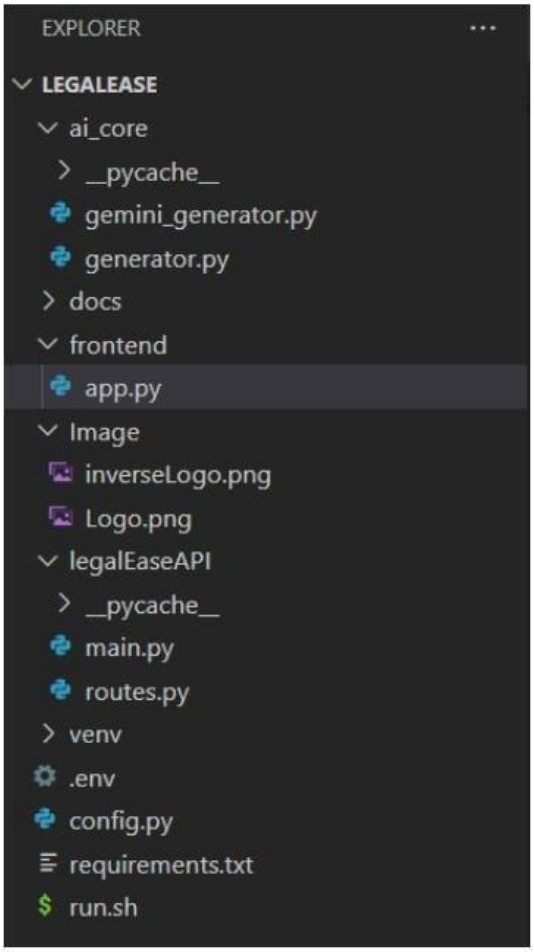
5. **Install Google Generative AI SDK**

pip install google-generativeai

6. **Install Additional Libraries**

pip install python-docx fpdf Pillow requests python-dotenv

7. **Set Up Application Structure**



MILESTONE 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities

1. Legal Document Generation

Functionality: Generate legal documents (e.g., contracts, agreements) based on user inputs (document type, parties, terms, and dates).

```python
# routes.py
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}
```

```python
# gemini_generator.py
def generate_document(self, document_type, parties, terms, dates):
    prompt = (
        f"Generate a comprehensive legal document titled '{document_type}'\n"
        f"Involved parties: {parties}\n"
        f"Effective Date: {dates}\n"
        f"Terms and conditions: {terms}\n"
        f"Ensure formal legal structure with multiple sections and legal clauses."
    )
    response = self.model.generate_content(prompt)
    return response.text
```

2. Document Output Formatting

Functionality: Convert raw AI-generated text into structured .docx, .pdf, .txt files with legal formatting, logos, tables, and footers.

- format_docx(text, doc_type)
- format_pdf(text, doc_type)

- format_html_preview(text)

These use:

- **docx** for Microsoft Word formatting (includes logo, Times New Roman font, terms as table, footer).
- **fpdf** for clean, branded PDF documents with headers/footers.
- **HTML Preview** with Streamlit styling.

---

3. User Interaction & Recommendations

Functionality: Accept user preferences through the Streamlit interface and reflect them in the AI-generated legal content.

```
# app.py - Streamlit UI
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions")
dates = st.text_input("Effective Date")

# Send to backend
response = requests.post("http://localhost:8000/generate", json={
    "document_type": document_type,
    "parties": parties,
    "terms": terms,
    "dates": dates
})
```

4. Editable Document & Download

Functionality: Provides editable text area, allows download in .txt, .docx, .pdf formats.

```
# app.py
if st.session_state.get("show_edit"):
    edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
    st.session_state.generated_text = edited_text

# Downloads
st.download_button("📄 Download as .TXT", data=generated_text, ...)
st.download_button("📝 Download as .DOCX", data=format_docx(...), ...)
st.download_button("📕 Download as .PDF", data=format_pdf(...), ...)
```

Activity 2.2: Implement the FastAPI Backend

1. Define Routes in FastAPI (Instead of Flask)

```
# main.py
app.include_router(router)

# routes.py
@router.post("/generate")  # Route for generating legal document
```

2. Process User Input

Handled via Pydantic & Streamlit Forms:

```
# routes.py
class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

```
# app.py
# Streamlit collects input → FastAPI handles it
response = requests.post("http://localhost:8000/generate", json={...})
```

3. **Integrate Groq (Gemini) API Calls**

```
# gemini_generator.py
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

self.model = genai.GenerativeModel(model_name)
response = self.model.generate_content(prompt)
```